📄 جزوه سیستم عامل - فصل ۷: مثال‌های همگامی

(بخش اول: مسائل کلاسیک همگامی)

۱. مقدمه فصل ۷

در فصل قبل ابزارهایی مثل سمافور، میوتکس و مانیتور را یاد گرفتیم. در این فصل می‌خواهیم ببینیم این ابزارها چطور در عمل برای حل مسائل واقعی استفاده می‌شوند. این مسائل "کلاسیک" هستند، یعنی تقریباً هر سیستم جدیدی که ساخته می‌شود، با این مسائل تست می‌شود.

۲. مسئله اول: بافر محدود (Bounded Buffer Problem)

(یادآوری از فصل‌های قبل)

ما یک انبار (Buffer) با ظرفیت محدود n داریم.

- **تولیدکننده (Producer):** داده تولید می‌کند و در بافر می‌گذارد.
- **مصرف‌کننده (Consumer):** داده را از بافر برمی‌دارد.

🛠️ ابزارهای همگامی مورد نیاز:

ما از ۳ سمافور استفاده می‌کنیم:

1. mutex (مقدار اولیه ۱): برای حفاظت از بافر (که همزمان دو نفر دست نزنند).
2. empty (مقدار اولیه n): تعداد جاهای خالی را می‌شمارد.
3. full (مقدار اولیه ۰): تعداد جاهای پر را می‌شمارد.

✓ الگوریتم حل مسئله:

الف) کد تولیدکننده (Producer):

```
} while (true)
/* تولید یک آیتم جدید */

wait(empty); // ۱ آیا جای خالی هست؟ (یکی از خالی‌ها کم کن)
wait(mutex); // ۲ قفل بافر را بگیر (که مصرف‌کننده همزمان نیاید)

/* آیتم را در بافر بگذار */

signal(mutex); // ۳ قفل را آزاد کن
signal(full); // ۴ اعلام کن که یک جای پر اضافه شد
{
```

ب) کد مصرف‌کننده (Consumer):

```
} while (true)
wait(full); // ۱ آیا چیزی برای خوردن هست؟ (یکی از پرها کم کن)
wait(mutex); // ۲ قفل بافر را بگیر

/* آیتم را از بافر بردار */

signal(mutex); // ۳ قفل را آزاد کن
signal(empty); // ۴ اعلام کن که یک جای خالی اضافه شد

/* آیتم را مصرف کن */
{
```

نکته مهم: دقت کنید که wait(empty) و wait(full) باید **خارج** از ناحیه mutex باشند، وگرنه ممکن است باعث بن‌بست (Deadlock) شود.

۳. مسئله دوم: خوانندگان - نویسندگان (Readers-Writers Problem)

فرض کنید یک دیتابیس مشترک داریم.

- **خوانندگان (Readers):** فقط می‌خوانند. (چند نفر همزمان می‌توانند بخوانند و مشکلی نیست).
- **نویسندگان (Writers):** می‌نویسند. (وقتی کسی می‌نویسد، **هیچ کس** دیگر نباید بخواند یا بنویسد).

چالش اصلی:

باید کاری کنیم که نویسنده دسترسی **انحصاری (Exclusive)** داشته باشد، اما خوانندگان بتوانند **اشتراکی (Shared)** استفاده کنند.

🛠 ابزارهای مورد نیاز:

1. rw_mutex (مقدار اولیه ۱): قفل اصلی برای نویسنده (یا اولین خواننده).
2. mutex (مقدار اولیه ۱): قفل کمکی برای تغییر متغیر read_count.
3. int read_count = 0: تعداد خوانندگان فعال.

✓ الگوریتم حل (نسخه اول - اولویت با خوانندگان):

الف) کد نویسنده (Writer):

خیلی ساده است! فقط باید قفل اصلی را بگیرد.

```
} while (true)
wait(rw_mutex); // قفل را بگیر (اگر کسی نیست)

/* نوشتن در دیتابیس */

signal(rw_mutex); // قفل را آزاد کن
{
```

ب) کد خواننده (Reader):

کمی پیچیده‌تر است. "اولین" نفری که وارد می‌شود باید چراغ را روشن کند (قفل نویسنده را بگیرد) و "آخرین" نفری که خارج می‌شود باید چراغ را خاموش کند.

```
} while (true)
wait(mutex);          // حفاظت از متغیر read_count
++read_count;         // من آمدم!
if (read_count == 1)  // اگر من اولین نفرم
wait(rw_mutex);   // راه نویسنده را ببند (قفل اصلی را بگیر)
signal(mutex);

/* خواندن دیتابیس (همزمان با بقیه خوانندگان) */

wait(mutex);          // حفاظت از متغیر read_count
--read_count;         // من دارم می‌روم
if (read_count == 0)  // اگر من آخرین نفرم
signal(rw_mutex); // راه نویسنده را باز کن
signal(mutex);
{
```

تحلیل: در این روش، تا وقتی حتی ۱ خواننده باشد، نویسنده پشت در می‌ماند. ممکن است نویسنده دچار **قحطی‌زدگی (Starvation)** شود.

۴. مسئله سوم: فیلسوفان خوش‌خوراک (Dining Philosophers)

(یک مسئله کلاسیک و کمی طنزآمیز!)

۵ فیلسوف دور یک میز گرد نشسته‌اند. بین هر دو نفر، **یک چوبک (Chopstick)** قرار دارد. (کلاً ۵ چوبک).

- فیلسوف یا فکر می‌کند یا غذا می‌خورد.
- برای غذا خوردن، باید **دو چوبک** (یکی سمت چپ و یکی سمت راست) را داشته باشد.

❌ راه حل ساده (و اشتباه) با سمافور:

هر چوبک یک سمافور است.

```
wait(chopstick[i]);              // چوبک چپ را بردار
wait(chopstick[(i + 1) % 5]);    // چوبک راست را بردار
                                 /* بخور */
signal(chopstick[i]);            // چپ را بگذار
signal(chopstick[(i + 1) % 5]); // راست را بگذار
```

مشکل این روش: اگر همه همزمان چوبک سمت چپشان را بردارند، هیچ چوبکی روی میز نمی‌ماند برای دست راست. همه منتظر سمت راست می‌مانند و **بن‌بست (Deadlock)** رخ می‌دهد!

✅ راه حل‌های جلوگیری از بن‌بست:

1. حداکثر ۴ نفر دور میز بنشینند (همیشه یک صندلی خالی باشد).
2. فیلسوف فقط وقتی چوبک بردارد که **هر دو** آزاد باشند (در یک بخش بحرانی).
3. **راه حل نامتقارن:** فیلسوف‌های فرد اول چپ را بردارند، فیلسوف‌های زوج اول راست را.

✅ راه حل با مانیتور (Monitor Solution):

این روش تضمین می‌کند که بن‌بست رخ ندهد.

- سه وضعیت برای فیلسوف تعریف می‌کنیم: THINKING (فکر کردن)، HUNGRY (گرسنه)، EATING (در حال خوردن).

- فیلسوف فقط وقتی می‌تواند EATING شود که همسایه‌های چپ و راستش EATING **نباشند**.
- توابع مانیتور:
  - pickup(i): اعلام گرسنگی می‌کند. اگر همسایه‌ها می‌خوردند، منتظر می‌ماند (Wait).
  - putdown(i): غذا خوردن تمام شد. وضعیت را به THINKING می‌برد و چک می‌کند آیا همسایه‌ها منتظرند یا نه (Signal).

(کد مانیتور کمی طولانی است که در صفحه ۴۱۰ کتاب آمده، اما منطق اصلی همین ۳ خط بالاست).

پایان بخش اول فصل ۷